Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Разбор инцидента

Источники, доказательства и отчёт · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Когда применять 4

3 Перед началом 5

4 Общий порядок 6

5 Источники сведений 7

6 Сохранение доказательств 11

6.1 1. Защищённый каталог 11

6.2 2. Журналы и состояние контейнеров 11

6.3 3. Журналы из базы 11

6.4 4. Копия базы на момент разбора 11

6.5 5. Контрольные суммы 11

6.6 6. Подробный журнал запросов на время разбора 12

7 Подбор пароля 13

7.1 Разбор 13

7.2 Сдерживание 13

8 Кража сеанса 15

8.1 Разбор 15

8.2 Сдерживание 15

9 Несанкционированное изменение данных 17

9.1 Разбор 17

9.2 Сдерживание и исправление 18

10 Инцидент доступности 19

11 Инцидент данных 20

12 Проверка 21

13 Если не получилось 22

14 Шаблон отчёта об инциденте 23

15 Запись в журнал эксплуатации 24

1 О документе

Регламент разбора инцидента «Планировщика отпусков» 1.0.0 (E52-04): инциденты информационной безопасности (подбор пароля, кража сеанса, несанкционированное изменение данных), инциденты доступности и данных; источники сведений и как их связать; сохранение доказательств; шаблон отчёта. Механизмы, на которые опирается разбор: журнал входов, сеансы и блокировка — ADR-015; журнал изменений в одной транзакции с изменением — ADR-006; код запроса и журнал сервера — `docs/runbooks/diagnostics.md`. Работа кадровой службы с журналом аудита, блокировками и сеансами — «Руководство администратора», разделы «Журнал аудита» и «Типовые операции».

Команды выполняются на узле стенда из каталога, где лежат файлы compose и `.env`. Время во всех журналах — UTC; в хронологию отчёта записывайте UTC и местное время.

2 Когда применять

ТИП	ПРИЗНАКИ	РАЗДЕЛ
ИБ: подбор пароля	Всплеск «неверный пароль», «неизвестный логин», «отказ по частоте», «блокировка» в журнале входов; сотрудник получил уведомление о блокировке, которую не вызывал	«Подбор пароля»
ИБ: кража сеанса	Запись «повтор токена — сеанс завершён» (`refresh_reuse`) в журнале входов; в списке сеансов сотрудника чужое устройство или адрес	«Кража сеанса»
ИБ: несанкционированное изменение данных	Изменения, о которых автор не знает; служебные права без заявки; изменения без записи в журнале изменений	«Несанкционированное изменение данных»
Доступность	Критичное оповещение (`alerts.md`) или простой дольше 15 минут	«Инцидент доступности»
Данные	Потеря или искажение данных, восстановление с потерей, нет годной копии	«Инцидент данных»

Разбор проводится и тогда, когда работа уже восстановлена: инцидент закрывается отчётом.

3 Перед началом

ЧТО НУЖНО	ПОДРОБНОСТИ
Кто	Руководитель разбора — служба ИБ (инциденты ИБ) или ответственный за систему (остальные); администратор стенда — сбор сведений и действия на узле; кадровая служба — блокировки, сеансы, пароли, связь с сотрудниками; разработчики — по запросу
Доступ	SSH к узлу стенда, право запускать `docker`; учётная запись кадровой службы или администратора в интерфейсе; защищённый каталог для доказательств
Время	Сдерживание — сразу; доказательства — в первый час: журналы контейнеров пропадают при пересоздании контейнера, а токены завершённых сеансов удаляются задачей `data-retention` через сутки
Утилиты	`jq`, `sha256sum`

Задайте переменные и функции один раз в сеансе:

```
C="docker compose"                                                    # стенд без
реплики
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"     # стенд с репликой
и архивом WAL
db() { $C exec -T postgres psql -U vacation -d vacation -c "$1"; }
csv() { $C exec -T postgres psql -U vacation -d vacation -c "\copy ($1) to stdout with csv
header"; }
```

После переключения на реплику основной сервер — `postgres-replica`: подставьте его в `db()` и `csv()`.

4 Общий порядок

1. **Зарегистрируйте.** Номер, время обнаружения (UTC и местное), кто сообщил, признаки, предварительный тип и приоритет по классификации «Описания процессов жизненного цикла»: критический — система недоступна, потеря или искажение данных, уязвимость с риском несанкционированного доступа.
2. **Сохраните доказательства** (раздел ниже) — до перезапусков контейнеров и правок данных. Если ущерб продолжается (идут изменения от чужого имени), сначала выполните сдерживание из раздела типа инцидента, затем сразу — сохранение.
3. **Сдержите.** Действия раздела типа инцидента.
4. **Разберите.** Хронология по источникам: журнал входов → сеансы → журнал изменений → журнал сервера и nginx по коду запроса и времени.
5. **Устраните и восстановите.** Права, пароли, данные — через интерфейс, чтобы исправления попали в журнал изменений; восстановление базы — `backup-restore.md`.
6. **Отчёт** по шаблону, меры с ответственными и сроками.
7. **Запись** в журнал эксплуатации.

Если затронуты персональные данные сотрудников (неправомерный доступ, передача, утечка), сразу сообщите ответственному за обработку персональных данных организации: организация — оператор этих данных (проект лицензионного договора, п. 7.2) и при неправомерной или случайной передаче (предоставлении, распространении, доступе) персональных данных уведомляет Роскомнадзор в течение 24 часов, а о результатах расследования — в течение 72 часов (ч. 3.1 ст. 21 Федерального закона № 152-ФЗ). Решение об уведомлении принимает ответственный за обработку персональных данных.

5 Источники сведений

ИСТОЧНИК	ЧТО СОДЕРЖИТ	КАК ПОЛУЧИТЬ	СКОЛЬКО ХРАНИТСЯ
Журнал входов (`login_events`)	Каждая попытка входа и повтор refresh-токена: время, сотрудник, логин, результат, способ (`password`, `oidc`, `refresh`), адрес, браузер	«Журнал аудита → Входы» (фильтры: логин, результат, «Только неудачные», период); `GET /api/v1/login-events?login=&result=&failed=true&from=&to=` (даты); SQL	Срок хранения журнала входов из правил организации (по умолчанию 365 дней), ежедневная очистка
Журнал изменений (`audit_events`)	Каждое изменение данных через приложение: время, автор (`actor_id`), действие, тип и идентификатор объекта, значения до и после	«Журнал аудита → Изменения» (фильтр по типу объекта); `GET /api/v1/audit-events?employee=<id автора>&entity=<тип>`; SQL	Не удаляется. Изменения напрямую в БД (psql) в него не попадают
Сеансы (`refresh_tokens`)	Цепочка токенов каждого сеанса: `session_id`, начало, способ входа, время каждого обновления с адресом и браузером, отзыв	SQL; сотрудник видит свои сеансы в профиле, кадровая служба — их число в карточке	Истёкшие токены и токены завершённых сеансов удаляются через сутки (`data-retention`)

ИСТОЧНИК	ЧТО СОДЕРЖИТ	КАК ПОЛУЧИТЬ	СКОЛЬКО ХРАНИТСЯ
Журнал сервера	Строки JSON: `ts`, `level`, `msg`, `reqId`; запросы — метод, маршрут, код, длительность, `user` (идентификатор сотрудника), `ip`; `"msg":"unhandled error"` со стеком. При `REQUEST_LOG=slow` — только медленные и 5xx, при `all` — каждый запрос. Паролей, токенов и тел запросов нет	`$C logs --no-log-prefix server`	Пока существует контейнер: пересоздание (`up -d` после правки `.env`, обновление) его очищает; ротация — настройками Docker узла
Журнал nginx	Каждый запрос: адрес, время, строка запроса, код, браузер, длительность, `rid=`	`$C logs --no-log-prefix web`	Как у журнала сервера
Журнал PostgreSQL	Ошибки сервера БД, операторы дольше `PG_SLOW_MS` (с текстом и значениями параметров)	`$C logs postgres`	Как у журнала сервера
Журнал прокси организации	Настоящие адреса клиентов и TLS	Средствами организации	По правилам организации
Метрики	Ошибки, задержки, соединения, история оповещений (ряд `ALERTS`)	Prometheus стенда, Grafana	30 дней в Prometheus стенда
Состояние контейнеров	Код завершения, нехватка памяти, время старта и останова	`docker inspect`	Пока существует контейнер

Ограничения источников: адрес в журнале nginx — адрес соединения (за прокси организации это адрес прокси; `X-Forwarded-For` nginx не пишет); адрес в журналах входов и сервера определяется с учётом `TRUST_PROXY` — при неверной настройке у всех один адрес прокси. Строка запроса в журнале nginx содержит одноразовые параметры (билет потока событий, код единого входа) — храните журнал как конфиденциальный.

Ключи для связи источников:

КЛЮЧ	ГДЕ ВСТРЕЧАЕТСЯ
Код запроса `X-Request-Id`	Заголовок ответа и поле `requestId` ошибки; «Код запроса: 1a2b3c4d» в сообщении интерфейса (первые 8 символов); `rid=` в журнале nginx; `reqId` в журнале сервера
Идентификатор сотрудника	`user` в журнале сервера; `actor_id` журнала изменений; `employee_id` журнала входов; `user_id` сеансов
Время	UTC во всех источниках; `audit_events.ts` и времена в `refresh_tokens` хранятся без пояса, в UTC

Результаты журнала входов:

RESULT	В ИНТЕРФЕЙСЕ	ЗНАЧЕНИЕ
`success`	вход	Вход по паролю
`sso_success`	единый вход	Вход через провайдера единого входа
`bad_password`	неверный пароль	Неверный пароль известного логина. Запись без адреса и браузера — неверный старый пароль при смене пароля в уже открытом сеансе (с 1.0.0 считается неудачной попыткой)
`unknown_login`	неизвестный логин	Логина нет или сотрудник уволен
`locked`	блокировка	Достигнут порог неудачных попыток — учётная запись заблокирована; или попытка входа во время блокировки
`rate_limited`	отказ по частоте	Больше 10 попыток в минуту с одного адреса для одного логина или 60 с одного адреса

RESULT	В ИНТЕРФЕЙСЕ	ЗНАЧЕНИЕ
`password_disabled`	пароль отключён	Вход по паролю при `LOCAL_LOGIN=false` у сотрудника без прав администратора
`sso_unknown`	единый вход — сотрудник не найден	Провайдер подтвердил вход, а сотрудника нет или он уволен
`refresh_reuse`	повтор токена — сеанс завершён	Предъявлен refresh-токен, использованный больше 30 секунд назад: сервер завершил сеанс целиком

6 Сохранение доказательств

6.1 1. Защищённый каталог

```
I=/srv/incident/$(date -u +%Y%m%d)-<номер>
mkdir -p "$I" && chmod 700 "$I"
```

6.2 2. Журналы и состояние контейнеров

До любого перезапуска или пересоздания контейнеров:

```
$C logs --no-log-prefix --timestamps server   > "$I/server.log"
$C logs --no-log-prefix --timestamps web      > "$I/web.log"
$C logs --no-log-prefix --timestamps postgres > "$I/postgres.log"
$C --profile full ps -a > "$I/ps.txt"
docker inspect $($C --profile full ps -aq) > "$I/inspect.json"
```

6.3 3. Журналы из базы

Период — с запасом до начала инцидента (ниже — 30 дней). Токены выгружаются без хешей.

```
csv "select * from login_events where ts > now() - interval '30 days'" >
"$I/login_events.csv"
csv "select * from audit_events where ts > now() - interval '30 days'" >
"$I/audit_events.csv"
csv "select id, user_id, session_id, session_started_at, method, created_at, revoked_at,
expires_at, ip, user_agent from refresh_tokens" > "$I/refresh_tokens.csv"
csv "select employee_id, last_login_at, failed_logins, locked_until from login_state" >
"$I/login_state.csv"
```

Ожидаемый результат: файлы CSV с заголовком; `refresh_tokens.csv` снят в первые сутки — позже токены завершённых сеансов будут удалены.

6.4 4. Копия базы на момент разбора

```
$C exec backup sh -c 'pg_dump -Fc -f /backups/vacation-incident-$(date +%Y%m%d-%H%M).dump'
$C cp backup:/backups/vacation-incident-<дата-время>.dump "$I/"
```

6.5 5. Контрольные суммы

```
(cd "$I" && sha256sum * > SHA256SUMS)
```

Запишите в отчёт, кто и когда собрал каждый файл. Снимки панелей Grafana за период инцидента с запасом в час положите туда же. Каталог содержит персональные данные — доступ только участникам разбора; передача в службу ИБ — её средствами.

6.6 6. Подробный журнал запросов на время разбора

Если атака продолжается и нужен каждый запрос с сотрудником и адресом, включите `REQUEST_LOG=all` — только после шага 2, потому что пересоздание очищает журнал контейнера:

```
# в .env: REQUEST_LOG=all
$C --profile full up -d server
```

После разбора верните `REQUEST_LOG=slow` (или удалите строку) и снова `$C --profile full up -d server`, предварительно выгрузив журнал.

7 Подбор пароля

Защита на стороне планировщика: ограничение частоты (10 попыток в минуту на пару «адрес + логин», 60 — на адрес) и временная блокировка учётной записи после серии неудачных попыток (правила организации: порог — по умолчанию 10, время — 15 минут); сотрудник получает уведомление о блокировке.

7.1 Разбор

```
# неудачные попытки за сутки по адресам
db "select ip, count(*) as attempts, count(distinct login) as logins, min(ts), max(ts) from
login_events where result in ('bad_password','unknown_login','locked','rate_limited') and ts
> now() - interval '24 hours' group by ip order by attempts desc limit 20"
# учётные записи под атакой
db "select login, count(*) filter (where result = 'bad_password') as bad, count(*) filter
(where result = 'locked') as locked, count(distinct ip) as addresses from login_events where
ts > now() - interval '24 hours' group by login having count(*) filter (where result not in
('success','sso_success')) > 5 order by bad desc limit 20"
# успешный вход после неудачных — пароль, возможно, подобран
db "select s.ts, s.login, s.ip, left(s.user_agent, 60) as browser from login_events s where
s.result = 'success' and s.ts > now() - interval '24 hours' and exists (select 1 from
login_events f where lower(f.login) = lower(s.login) and f.result in
('bad_password','locked') and f.ts between s.ts - interval '1 hour' and s.ts) order by s.ts"
```

КАРТИНА	ТОЛКОВАНИЕ
Один адрес, много логинов, `unknown_login` и `rate_limited`	Перебор логинов с одного адреса
Много адресов, один логин, `bad_password` и `locked`	Распределённый подбор пароля одной учётной записи — сдерживает блокировка
Все попытки — с одного адреса прокси организации	Неверный `TRUST_PROXY` (руководство по развёртыванию, «Обратный прокси»); настоящие адреса — в журнале прокси
Успешный вход после серии неудач с того же адреса	Пароль мог быть подобран — уточните у сотрудника, был ли это он
`bad_password` и `locked` без адреса	Неверный старый пароль в форме смены пароля внутри открытого сеанса — кто-то в чужом сеансе подбирает пароль: разбирайте как кражу сеанса

7.2 Сдерживание

1. Адреса источника блокируются на прокси или межсетевом экране организации — в планировщике блокировки по адресу нет.

2. Учётные записи с возможно подобранным паролем: кадровая служба выдаёт новый одноразовый пароль (ключ в списке сотрудников — все сеансы завершаются) и сообщает сотруднику.
3. Массовые блокировки как отказ в обслуживании: блокировки временные; после блокировки источника кадровая служба снимает их («Учётная запись → Снять блокировку»).
4. Если вход по паролю сотрудникам не нужен — единый вход и `LOCAL_LOGIN=false` (руководство, «Единый вход»): пароль останется только у администраторов.

Проверка. Новых неудачных попыток с адресов источника нет; затронутые сотрудники сменили пароль; действия от их имени за период проверены по журналу изменений (раздел «Несанкционированное изменение данных»).

8 Кража сеанса

Refresh-токен меняется при каждом обновлении сеанса. Если уже использованный токен предъявлен повторно позже 30 секунд, сервер считает его украденным: завершает сеанс целиком и пишет в журнал входов `refresh_reuse` (ADR-015). Так проявляется копия cookie на другом устройстве (вредоносная программа, общий компьютер); изредка — профиль браузера, восстановленный из копии.

8.1 Разбор

```
# события повтора токена
db "select ts, employee_id, login, ip, left(user_agent, 80) as browser from login_events
where result = 'refresh_reuse' order by ts desc limit 20"
# сеансы сотрудника: каждая строка — одно обновление с адресом и браузером того, кто обновлял
db "select session_id, session_started_at, method, created_at, revoked_at, ip,
left(user_agent, 60) as browser from refresh_tokens where user_id = <id> order by
session_started_at, created_at"
# входы сотрудника за неделю
db "select ts, result, method, ip, left(user_agent, 60) as browser from login_events where
employee_id = <id> and ts > now() - interval '7 days' order by ts"
# что сделано от его имени
db "select ts, action, entity, entity_id from audit_events where actor_id = <id> and ts >
now() - interval '7 days' order by ts"
```

Толкование:

1. В цепочке одного `session_id` сменились адрес или браузер — сеансом пользовались с другого устройства; время смены — начало компрометации.
2. Адрес события `refresh_reuse` — того, кто предъявил старый токен. Привычный адрес сотрудника — похититель обновил сеанс первым (его адрес — в цепочке); незнакомый адрес — похититель предъявил украденную копию.
3. Записи журнала изменений между началом компрометации и `refresh_reuse` — действия похитителя от имени сотрудника; их разбор — в разделе «Несанкционированное изменение данных».
4. В журнале nginx в это время — `POST /api/v1/auth/refresh` с кодом `401`; адрес за прокси организации — в журнале прокси.

8.2 Сдерживание

1. Сеанс уже завершён сервером. Кадровая служба в карточке сотрудника нажимает «Завершить все сеансы» — другие сеансы тоже могли быть скопированы — и, если пароль мог быть перехвачен, выдаёт новый одноразовый пароль.
2. Служба ИБ проверяет устройство сотрудника.
3. Сотрудник проверяет список своих сеансов в профиле после повторного входа.

Проверка. Новых `refresh_reuse` по сотруднику нет; в его сеансах только его устройства; действия от его имени за период подтверждены или исправлены.

9 Несанкционированное изменение данных

Каждое изменение через приложение записывается в журнал изменений в той же транзакции, что и само изменение (ADR-006), с автором и значениями до и после. Изменения, сделанные напрямую в базе, в журнал не попадают.

9.1 Разбор

```
# история объекта; entity: vacation_request, employee, department, project, org_settings,
delegation, planning_campaign, schedule_approval, vacation_notice, calendar, import, report
db "select a.ts, e.login as actor, a.action, a.before, a.after from audit_events a left join
employees e on e.id = a.actor_id where a.entity = 'vacation_request' and a.entity_id = '<id>'
order by a.ts"
# все действия автора за период (время — UTC)
db "select ts, action, entity, entity_id from audit_events where actor_id = <id> and ts
between '<с>' and '<по>' order by ts"
# выдача прав администратора и кадровой службы
db "select a.ts, e.login as actor, a.action, a.entity_id as employee_id, a.before->>'isAdmin'
as was_admin, a.after->>'isAdmin' as admin, a.before->>'isHr' as was_hr, a.after->>'isHr' as
hr from audit_events a left join employees e on e.id = a.actor_id where a.entity = 'employee'
and a.action in ('employee.created','employee.updated') and (a.after->>'isAdmin' = 'true' or
a.after->>'isHr' = 'true') and ((a.before->>'isAdmin') is distinct from (a.after->>'isAdmin')
or (a.before->>'isHr') is distinct from (a.after->>'isHr')) order by a.ts desc limit 50"
# сбросы паролей, снятие блокировок, завершение сеансов, правила организации
db "select a.ts, e.login as actor, a.action, a.entity_id from audit_events a left join
employees e on e.id = a.actor_id where a.action in
('employee.password_reset','employee.unlocked','employee.sessions_revoked','settings.updated'
) order by a.ts desc limit 50"
```

Затем установите, действовал ли сам автор:

1. Журнал входов и сеансы автора около времени изменения (запросы раздела «Кража сеанса»): чужой адрес или браузер — учётная запись или сеанс скомпрометированы.
2. Запросы автора в журнале сервера (каждый запрос — только при `REQUEST_LOG=all`, иначе медленные и 5xx) и строки nginx того же кода запроса:

```
$C logs --no-log-prefix server | jq -cR 'fromjson? | select(.user == <id>) | {ts, method,
route, status, ip, reqId}'
$C logs --no-log-prefix web | grep 'rid=<код запроса>'
```

3. Данные объекта не совпадают со значением «после» последней записи журнала изменений — изменение сделано в обход приложения. Доступ к базе есть у тех, кто может выполнять `docker` на узле стенда (пароль БД задан в `docker-compose.yml`); до версии 1.0.0 порт PostgreSQL был открыт на всех интерфейсах узла с паролем из репозитория (`update.md`, «Особые шаги 1.0.0»). Журналы входа на узел и действий администраторов проверяет служба ИБ средствами организации.

9.2 Сдерживание и исправление

1. Учётная запись или сеанс автора скомпрометированы — «Завершить все сеансы» и новый одноразовый пароль (кадровая служба).
2. Лишние служебные права снимаются в карточке сотрудника — изменение попадёт в журнал изменений.
3. Данные исправляются через интерфейс кадровой службой. Массовое искажение — восстановление на момент до него или из копии по `backup-restore.md`, по решению ответственного за систему.

Проверка. Права соответствуют заявкам; исправления видны в журнале изменений; кадровая служба подтвердила данные.

10 Инцидент доступности

1. Восстановите работу по `alerts.md` — это и есть сдерживание.
2. Соберите сведения: доказательства (шаги 1–2 раздела «Сохранение доказательств»), история оповещений — выражение `ALERTS{alertname=~"Vacation.*", alertstate="firing"}` во вкладке «Graph» Prometheus за период; состояние узла:

```
docker inspect --format '{{.Name}} {{.State.Status}} exit={{.State.ExitCode}} oom=
{{.State.OOMKilled}} started={{.State.StartedAt}} finished={{.State.FinishedAt}}' $($C --
profile full ps -aq)
uptime
journalctl -k --since "<начало, местное время узла>" | grep -iE 'out of memory|killed
process'
df -h "$(docker info -f '{{.DockerRootDir}}')"
$C logs --no-log-prefix --since <начало> server | jq -cR 'fromjson? |
select(.level=="error")' | head -50
$C logs --since <начало> postgres | grep -E 'FATAL|PANIC|ERROR' | head -50
```

3. Оцените влияние: длительность (от первого оповещения или жалобы до восстановления), недоступные функции, число ответов 5xx за период — `sum(increase(vp_http_requests_total{code=~"5.."}[<период>]))` в Prometheus.
4. Найдите первопричину по `diagnostics.md` («Симптом → метрика → причина → действие»).
5. Были потери данных (переключение на реплику, восстановление из копии) — дополнительно раздел «Инцидент данных».

Проверка. Причина подтверждена метрикой или журналом; меры против повторения назначены.

11 Инцидент данных

1. **Остановите источник.** Приостановите импорт и скрипт интеграции с кадровой системой, если искажение идёт оттуда; скомпрометированную учётную запись — по разделу «Несанкционированное изменение данных»; дефект кода, который продолжает портить данные, — решение ответственного об остановке `server`.
2. **Сохраните текущее состояние** — шаг 4 «Сохранения доказательств» — до любого восстановления.
3. **Определите объём** по журналу изменений:

```
db "select date_trunc('minute', a.ts) as minute, e.login as actor, a.action, a.entity,
count(*) from audit_events a left join employees e on e.id = a.actor_id where a.ts between
'<c>' and '<no>' group by 1, 2, 3, 4 order by 1"
db "select ts, actor_id, after from audit_events where action = 'import.applied' order by ts
desc limit 5"
```

4. **Выберите способ восстановления** (`backup-restore.md`):

СЛУЧАЙ	СПОСОБ
Несколько записей	Исправление через интерфейс кадровой службой
Массовое искажение, стенд с архивом WAL	Восстановление на момент до искажения: вся база или точечная выгрузка
Массовое искажение без архива WAL	Полное восстановление из копии `pg_dump`; изменения после копии повторяются по выгрузке журнала изменений
Годной копии нет	Тома не удаляйте и не перезаписывайте; немедленная эскалация ответственному за систему и разработчикам

5. **После восстановления** кадровая служба сверяет данные и повторяет потерянные изменения; учтите, что вместе с копией возвращаются сеансы, пароли и блокировки на её момент (`backup-restore.md`, «Полное восстановление»).

Проверка. Кадровая служба подтвердила данные; потерянный период и повторённые изменения перечислены в отчёте.

12 Проверка

Инцидент закрывается, когда:

1. Причина установлена и подтверждена источниками либо в отчёте указано, почему её установить нельзя.
2. Сдерживание действует: признаков повторения нет в течение суток (новых неудачных входов с адресов источника, `refresh_reuse`, неожиданных изменений, оповещений).
3. Данные, права, пароли и сеансы приведены в порядок; кадровая служба подтвердила данные.
4. Доказательства сохранены с контрольными суммами, отчёт согласован руководителем разбора, меры назначены с ответственными и сроками.
5. Временные настройки разбора сняты (`REQUEST_LOG` возвращён к `slow`).

13 Если не получилось

СИТУАЦИЯ	КОМУ
Признаки продолжающейся атаки после сдерживания	Служба ИБ, ответственный за систему — решение об ограничении доступа к стенду на прокси организации
Затронуты персональные данные	Ответственный за обработку персональных данных организации — сразу
Причина в коде планировщика (обход прав, искажение данных)	Разработчики (поддержка правообладателя) — как дефект критического приоритета, с выдержками из журналов без персональных данных, где возможно
Нет годной копии, восстановление не удалось	Ответственный за систему, администратор БД, разработчики

14 Шаблон отчёта об инциденте

```
# Отчёт об инциденте <номер>

| Поле | Значение |
|---|---|
| Номер и дата отчёта | |
| Тип | ИБ: подбор пароля / кража сеанса / изменение данных · доступность · данные |
| Приоритет | Критический / высокий / средний / низкий |
| Обнаружен | Когда (UTC и местное), кем, как: оповещение, жалоба, журнал |
| Период инцидента | Начало — окончание (UTC) |
| Версия планировщика | |
| Руководитель разбора и участники | |

## Хронология

| Время (UTC) | Местное | Событие | Источник |
|---|---|---|---|

## Влияние

- Затронутые пользователи и функции
- Длительность простоя
- Потерянные или искажённые данные: объём, период
- Затронутые персональные данные: какие, чьи

## Причина

- Непосредственная причина
- Первопричина
- Почему не обнаружено раньше

## Действия

| Время (UTC) | Действие | Кто | Результат |
|---|---|---|---|

## Доказательства

| Файл | Источник | SHA-256 | Кто и когда собрал |
|---|---|---|---|

## Уведомления

- Служба ИБ, ответственный за систему, руководство
- Ответственный за обработку персональных данных; Роскомнадзор (ч. 3.1 ст. 21 152-ФЗ):
требуется / не требуется, когда направлено

## Меры

| Мера | Ответственный | Срок | Статус |
|---|---|---|---|

## Выводы
```

15 Запись в журнал эксплуатации

ПОЛЕ	ПРИМЕР
Номер инцидента, тип, приоритет	2026-11-20-01, ИБ: кража сеанса, высокий
Время обнаружения и закрытия (UTC)	2026-11-20 08:14 — 2026-11-21 12:00
Руководитель разбора, участники	Служба ИБ; администратор стенда; кадровая служба
Кратко: причина и действия	Копия cookie на заражённом ноутбуке; сеансы завершены, пароль сброшен, устройство передано ИБ
Доказательства и отчёт	Каталог `/srv/incident/20261120-01`, `SHA256SUMS`; ссылка на отчёт